

TP4: Módulos de Kernel y Llamadas al Sistema

Materia: Sistemas de Computación **Grupo:** Electrotonto y Computarados **Integrantes:**

- Martina Juri
- Marcos Morán
- Francisco Gomez Neimann
- Cristian Arteaga

Docentes:

- Miguel Ángel Solinas
 - Javier Alejandro Jorge
-

Introducción

En este trabajo práctico se investigó el funcionamiento de los módulos del kernel en Linux, su carga y descarga dinámica, la diferencia entre espacio de usuario y espacio del kernel, y distintas herramientas relacionadas con el análisis del sistema operativo.

Un módulo del kernel es una porción de código que puede cargarse y descargarse dinámicamente en el núcleo del sistema operativo sin necesidad de reiniciar el sistema. Esto permite extender funcionalidades del kernel de manera flexible, por ejemplo agregando drivers, soporte para hardware o sistemas de archivos.

Durante el desarrollo del trabajo se compiló y cargó un módulo propio, se analizaron los mensajes del kernel utilizando `dmesg`, se inspeccionaron módulos cargados con `lsmod`, se investigó el contenido de `/dev` y se comparó un módulo desarrollado por nosotros con módulos reales del sistema Linux.

Además, se exploraron conceptos relacionados con Secure Boot y la firma de módulos del kernel, aunque la máquina virtual utilizada para esta primera parte del trabajo no poseía soporte EFI habilitado.

Preparación del entorno

Para realizar el trabajo se utilizó una máquina virtual con Ubuntu Linux. Se instalaron las herramientas necesarias para compilar módulos del kernel y trabajar con paquetes.

```
sudo apt update
sudo apt install build-essential checkinstall linux-headers-$(uname -r)
```

También se verificó la versión del kernel utilizada:

```
uname -r
```

Resultado:

```
6.8.0-48-generic
```

Compilación del módulo

Se utilizó el módulo de ejemplo provisto por la cátedra dentro del directorio `part1/module`.

El módulo fue compilado utilizando `make`:

```
make
```

Esto generó el archivo:

```
mimodulo.ko
```

que corresponde al módulo compilado listo para cargarse en el kernel.

Carga del módulo en el kernel

Para cargar el módulo se utilizó:

```
sudo insmod mimodulo.ko
```

Luego se verificaron los mensajes del kernel mediante:

```
sudo dmesg | tail
```

Obteniendo:

```
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
• % sudo insmod mimodulo.ko
[sudo] password for martina:
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
• % sudo dmesg | tail
[ 70.715790] 13:29:12.279695 main 7.0.20 r163906 started. Verbose level = 0
[ 70.717676] 13:29:12.281645 main vbglR3GuestCtrlDetectPeekGetCancelSupport: Supported (#1)
[ 98.859448] Initializing XFRM netlink socket
[ 98.926217] bridge: filtering via arp/ip/ip6tables is no longer available by default. Update your scripts to load br_netfilter if you need this.
[ 103.093804] systemd-journald[289]: /var/log/journal/9d9e86572f6949a9ba0fcb66ebef9a7b/user-1000.journal: Journal file uses a different sequence number ID, rotating.
[ 108.142255] ISO 9660 Extensions: Microsoft Joliet Level 3
[ 108.143816] ISO 9660 Extensions: RRIP_1991A
[ 119.400459] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
[ 183.473561] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 8 times, consider switching to WQ_UNBOUND
[ 326.462821] Modulo cargado en el kernel (Grupo Electrotonto y Computarados).
```

Esto indica que el módulo fue cargado correctamente y que la función de inicialización del módulo se ejecutó exitosamente.

Verificación de módulos cargados

Una vez cargado el módulo, se verificó su presencia mediante `lsmod`:

```
lsmod | grep mimodulo
```

Resultado:

```
mimodulo 12288 0
```

Esto indica:

- nombre del módulo
- tamaño ocupado en memoria
- cantidad de referencias activas

También se inspeccionó `/proc/modules`:

```
cat /proc/modules | grep mimodulo
```

Resultado:

```
mimodulo 12280 0 - Live 0x0000000000000000 (OE)
```

El sufijo `(OE)` indica que el módulo es:

- **O**: Out-of-tree → módulo externo al árbol oficial del kernel.
- **E**: unsigned/external → módulo externo no firmado.

Información del módulo utilizando modinfo

Se utilizó:

```
modinfo mimodulo.ko
```

Obteniendo:

```
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
• % lsmod | grep mimodulo
mimodulo                12288  0
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
• % cat /proc/modules | grep mimodulo
mimodulo 12288 0 - Live 0x0000000000000000 (OE)
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
• % modinfo mimodulo.ko
filename:                /home/martina/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module/mimodulo.ko
author:                  Catedra de SdeC
description:             Primer modulo ejemplo
license:                 GPL
srcversion:              6C8EF10184859FB68E61448
depends:
retpoline:              Y
name:                   mimodulo
vermagic:                6.8.0-48-generic SMP preempt mod unload modversions
```

El campo **vermagic** resulta especialmente importante, ya que indica la versión del kernel con la cual el módulo fue compilado. Si el módulo se compila utilizando headers incompatibles, el kernel puede rechazar su carga.

Comparación con un módulo real del sistema

Se buscaron módulos disponibles dentro del sistema mediante:

```
find /lib/modules/${uname -r} | grep '\.ko$' | head -20
```

Entre ellos se seleccionó el módulo:

```
/lib/modules/6.8.0-48-generic/misc/vboxvideo.ko
```

Luego se ejecutó:

```
modinfo /lib/modules/6.8.0-48-generic/misc/vboxvideo.ko
```

Resultado parcial:

```
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kenel-modules/part1/module
• % find /lib/modules/$(uname -r) | grep '\.ko$' | head -20
/lib/modules/6.8.0-48-generic/misc/vboxvideo.ko
/lib/modules/6.8.0-48-generic/misc/vboxsf.ko
/lib/modules/6.8.0-48-generic/misc/vboxguest.ko
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kenel-modules/part1/module
⊙ % mdinfo /lib/modules/6.8.0-48-generic/misc/vboxvideo.ko
zsh: command not found: mdinfo
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kenel-modules/part1/module
• % modinfo /lib/modules/6.8.0-48-generic/misc/vboxvideo.ko
filename:      /lib/modules/6.8.0-48-generic/misc/vboxvideo.ko
version:      7.0.20 r163906
license:      GPL and additional rights
description:   Oracle VM VirtualBox Graphics Card
author:       Oracle and/or its affiliates
srcversion:   594174D1EBD3FFA29D7A771
alias:        pci:v000080EEd0000BEEFsv*sd*bc*sc*i*
depends:       ttm,drm_ttm_helper
retpoline:    Y
name:         vboxvideo
vermagic:     6.8.0-48-generic SMP preempt mod_unload modversions
parm:         disabled:Disable automatic module loading (int)
parm:         modeset:Disable/Enable modesetting (int)
```

A diferencia de nuestro módulo simple, este módulo posee:

- dependencias con otros módulos
- parámetros configurables
- mayor cantidad de metadata
- funcionalidad real de driver gráfico para VirtualBox

Esto refleja la diferencia entre un módulo de ejemplo y un driver utilizado realmente por el sistema operativo.

Descarga del módulo

Para descargar el módulo del kernel se utilizó:

```
sudo rmmod mimodulo
```

Posteriormente se revisaron nuevamente los logs del kernel:

```
sudo dmesg | tail
```

Durante la descarga apareció un mensaje de error del kernel acompañado por registros internos y stack trace.

Finalmente se observó:

```
Modulo descargado del kernel.
```

Esto permitió observar cómo un error dentro de un módulo afecta directamente al kernel, ya que los módulos se ejecutan en espacio privilegiado y no poseen el aislamiento de memoria que tienen los programas normales de usuario.

```
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
* % sudo rmmod mimodulo
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
* % sudo dmesg | tail
[ 356.008409] Code: 04 25 28 00 00 00 48 89 45 c8 31 c0 48 8d 45 10 c7 45 b0 10 00 00 00 48 89 45 b8 48 8d 45 d0 48 89 45 c0 b8 10 00 00 00 0f 05 <89> c2 3d 00 f0 ff ff 77
1a 48 8b 45 c8 64 48 2b 04 25 28 00 00 00
[ 356.008412] RSP: 002b:00007e0ce63fe570 EFLAGS: 00000246 ORIG_RAX: 0000000000000010
[ 356.008416] RAX: ffffffffefda R8X: 00007e0ce63fe600 RCX: 00007e0ce6f24e1d
[ 356.008418] RDY: 00007e0ce63fe600 RSI: 00000000c0585607 RDI: 0000000000000003
[ 356.008420] RBP: 00007e0ce63fe5c0 R08: 00007e0ce63fe6a4 R09: 0000000000000000
[ 356.008422] R10: 00007e0cd4000030 R11: 0000000000000246 R12: 00007e0ce63fe6a4
[ 356.008424] R13: 000000000059f060 R14: 0000000000000002 R15: 00007e0ce63fe6a4
[ 356.008430] </TASK>
[ 356.008432] ---[ end trace ]---
[ 718.022139] Módulo descargado del kernel.
```

Diferencia entre un programa y un módulo

Programa de usuario

Un programa normal:

- se ejecuta en espacio de usuario
- utiliza bibliotecas como libc
- accede al hardware mediante llamadas al sistema
- posee memoria aislada y protegida

Si ocurre un error grave, normalmente solo se termina el proceso.

Módulo del kernel

Un módulo:

- se ejecuta en espacio del kernel
- posee privilegios totales
- accede directamente al hardware y memoria
- utiliza funciones internas del kernel

Errores dentro de un módulo pueden provocar:

- kernel oops
 - kernel panic
 - congelamiento del sistema
-

Espacio de usuario y espacio del kernel

Espacio de usuario

Es el entorno donde se ejecutan aplicaciones comunes. Los programas no poseen acceso directo al hardware y deben interactuar con el kernel mediante llamadas al sistema.

Espacio del kernel

Es el entorno donde se ejecuta el núcleo del sistema operativo y los módulos cargados. Posee acceso completo al hardware y a toda la memoria del sistema.

Todos los módulos comparten el mismo espacio de memoria del kernel, por lo que un error puede afectar a todo el sistema operativo.

Drivers y contenido de /dev

Se exploró el contenido del directorio `/dev` mediante:

```
ls /dev | head
```

Resultado:

```
autofs
block
bsg
btrfs-control
bus
cdrom
char
console
core
cpu
```

El directorio `/dev` contiene archivos especiales que representan dispositivos físicos y virtuales del sistema.

En Linux, muchos dispositivos se acceden como si fueran archivos.

Ejemplos:

- `/dev/sda` → disco rígido
- `/dev/cdrom` → lectora
- `/dev/null` → dispositivo virtual
- `/dev/random` → generador aleatorio

Los drivers del kernel son los encargados de gestionar estos dispositivos y exponer interfaces accesibles desde espacio de usuario.

```
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kenel-modules/part1/module
• % ls /dev | head
autofs
block
bsg
btrfs-control
bus
cdrom
char
console
core
cpu
```

Información de dispositivos del sistema

También se utilizó:

```
lsblk
```

Resultado parcial:

```
sda      79,4G
├─sda1    1M
├─sda2   513M /boot/efi
└─sda3   78,9G /
```

Esto permitió observar:

- discos detectados
- particiones
- puntos de montaje
- dispositivos virtuales utilizados por VirtualBox

```
martina@martina-VirtualBox ~/Electrotonto-y-Computarados/TP4/kenel-modules/part1/module
• % lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0       7:0      0 655,6M 1 loop /snap/ghidra/35
loop1       7:1      0   74M 1 loop /snap/core22/2411
loop2       7:2      0  49,3M 1 loop /snap/snapd/26865
sda          8:0      0  79,4G 0 disk
├─sda1       8:1      0    1M 0 part
├─sda2       8:2      0  513M 0 part /boot/efi
└─sda3       8:3      0  78,9G 0 part /
sr0         11:0     1    51M 0 rom  /media/martina/VBox_GAs_7.0.20
```

Secure Boot y EFI en la máquina virtual

Se intentó verificar el estado de Secure Boot mediante:


```
mokutil --sb-state
```

Resultado:

```
EFI variables are not supported on this system
```

Además se verificó:

```
ls /sys/firmware/efi
```

Resultado:

```
No such file or directory
```

Esto indica que la máquina virtual utilizada estaba funcionando en modo BIOS legacy y no utilizando UEFI con soporte EFI real.

Por este motivo:

- no fue posible probar Secure Boot completamente
- no fue posible registrar claves MOK
- el kernel permitió cargar módulos sin firma digital

Aun así, se investigó teóricamente el funcionamiento de Secure Boot y la firma de módulos del kernel.

```
martina@martina-VirtualBox: ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
% ls /sys/firmware/efi
ls: cannot access '/sys/firmware/efi': No such file or directory
martina@martina-VirtualBox: ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
% modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko
modinfo: ERROR: Module /lib/modules/6.8.0-48-generic/kernel/crypto/des_generic.ko not found.
martina@martina-VirtualBox: ~/Electrotonto-y-Computarados/TP4/kernel-modules/part1/module
% ls /dev/lp
```

¿Qué es checkinstall?

checkinstall es una herramienta que permite crear paquetes instalables (**.deb**) a partir de software compilado manualmente.

En lugar de utilizar:

```
sudo make install
```

se puede utilizar:

```
sudo checkinstall
```

Esto genera un paquete administrable por el sistema de paquetes de Linux, permitiendo:

- instalación limpia
- desinstalación sencilla
- seguimiento de archivos instalados

¿Qué ocurre si falta un driver?

Cuando un driver no está disponible:

- el sistema no puede inicializar correctamente el hardware
- el dispositivo puede aparecer como desconocido
- no se crea la entrada correspondiente en `/dev`
- el hardware queda inutilizable

Esto demuestra la importancia de los módulos del kernel y drivers dentro del sistema operativo.

Conclusión

Durante el desarrollo del TP4 se trabajó directamente con módulos del kernel Linux, observando cómo se compilan, cargan y descargan dinámicamente dentro del sistema operativo.

La práctica permitió comprender las diferencias fundamentales entre:

- programas de usuario
- módulos del kernel
- espacio de usuario
- espacio privilegiado del kernel

También se analizaron herramientas importantes del sistema como:

- `lsmod`
- `modinfo`
- `dmesg`
- `/proc/modules`
- `lsblk`

Además, se investigó el rol de Secure Boot y la importancia de las firmas digitales en módulos del kernel para prevenir la carga de código malicioso o rootkits.

Aunque la máquina virtual utilizada no contaba con soporte EFI ni Secure Boot habilitado, la experiencia permitió comprender el funcionamiento general de estos mecanismos de seguridad y su importancia en sistemas modernos.